

DAAB Knowledge backbone for AI agents

Code baseline: go `16dc548` · python `6966d4b`
web `907bc8b` · Status: pre-production

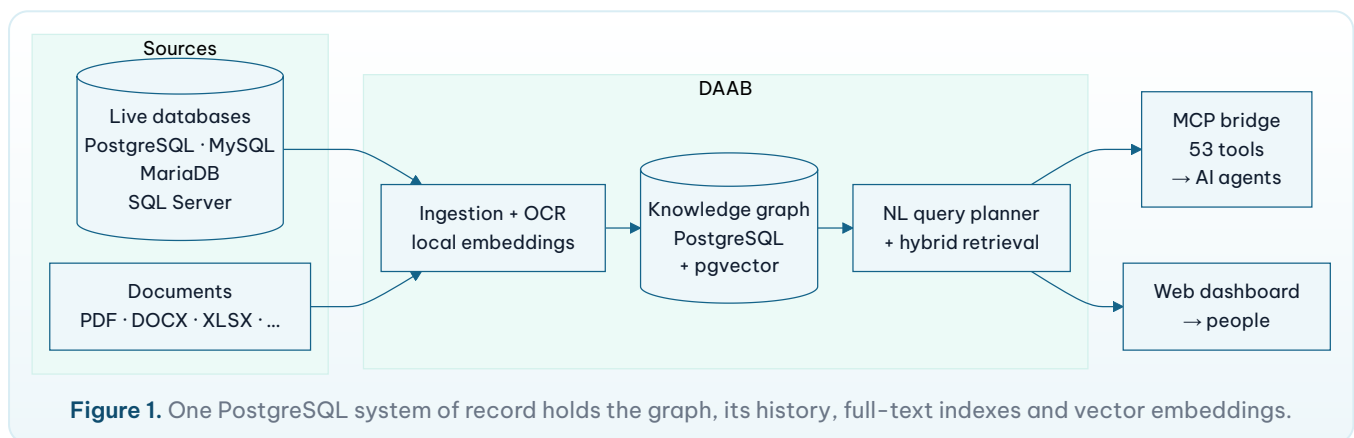
DAAB turns an organisation's **live business databases** and **documents** into one queryable knowledge graph, and serves it to AI agents through the Model Context Protocol (MCP) and to people through a web dashboard. Agents can search and cite documents, ask questions of live databases in plain language, remember facts across sessions and, under strict controls, write back.

53MCP TOOLS
WIRED TO THE API**251**

HTTP API ROUTES

4SOURCE DB
ENGINES**7**DOCUMENT
INGEST FORMATS**22**TYPED NODE
KINDS · 54
TABLES**3,668**TEST FUNCTIONS
(GO + PYTHON)

How it works



What is technically differentiated

- **The model proposes, code decides.** For agent database questions (MCP/API) the LLM returns a structured *query plan*, not SQL; the server validates it against the extracted schema and renders parameterised, read-only SQL (a READ ONLY session on PostgreSQL sources) with timeouts and row caps. Ambiguous questions return a clarification instead of a guess. The dashboard's agentic chat mode instead lets the model write `SELECT` statements, checked by a read-only validator with row caps.
- **Databases become a navigable graph.** Schema extraction turns tables, foreign keys and AI-detected implicit relationships into graph nodes that documents and agents can link to.
- **Ingest once, cite exactly.** Content-hash deduplication, section and chunk structure with line offsets, and Vietnamese-capable OCR, so agents quote the precise passage.
- **Hybrid retrieval and GraphRAG** on PostgreSQL alone, with no separate search or vector engine: full-text and vector arms fused by Reciprocal Rank Fusion, then expanded across cross-document similarity edges.
- **Staged entity resolution.** Local blocking, rules and a cross-encoder filter candidates before any LLM check; merges are recorded first and reversible. Internal benchmark: precision 1.000, recall 0.915.
- **Local embeddings, provider-agnostic LLMs.** Embedding and reranking run on CPU inside the platform; LLM providers are pluggable with failover and per-function model assignment.

Where it stands today

Capability	Status	Evidence
Knowledge graph, document ingestion, hybrid search, GraphRAG	LIVE	Exercised end-to-end with demo data: about 16.5k nodes, 35k edges and 12k embeddings
Live-database connectors and schema-to-graph	LIVE	PostgreSQL, MySQL, MariaDB and SQL Server sources connected
Natural-language data query for agents (MCP/API)	LIVE	LLM query plan → server validation → parameterised read-only SQL; plan cache
Dashboard chat over data sources (agentic mode)	LIVE	Model-written <code>SELECT</code> statements checked by a read-only validator with row caps
Entity resolution, controlled writes and playbooks	LIVE	Staged merge pipeline; opt-in audited inserts, approved playbooks (small volume)
Agent memory, code indexing, NL-query benchmark suite	BUILT	Implemented and reachable; not yet used with real data
Development stack (Docker Compose)	LIVE	Used daily for development and demonstrations
Self-hosted release bundle v1.0.0 · AWS cloud dev environment	BUILT	Bundle passed a clean-install smoke test; cloud dev environment defined, Go service deployed by CI/CD
Production deployment	PLANNED	Pre-production: no production tenant yet

Evidence counts come from the development/demo environment and show a capability has been exercised end-to-end; they are not production usage. The entity-resolution figures come from a small internal Vietnamese benchmark (30 entity groups, 86 duplicate pairs): precision 1.000, recall 0.915.

Technology

API & MCP	Go 1.25 standard library (<code>net/http</code> , <code>database/sql</code>), official MCP Go SDK, stdio and Streamable HTTP
Workers	Python 3.12, FastAPI, sentence-transformers (CPU), tree-sitter, Tesseract + RapidOCR
Dashboard	Next.js 16, React 19, TypeScript strict, 3D graph explorer
Data	PostgreSQL 16 + pgvector (HNSW), <code>pg_trgm</code> , unaccent; Redis job lists; 86 migrations
Quality	2,753 Go + 915 Python test functions (943 Python tests collected); Go CI with race detector and project-isolation gate

Security and data posture

- **Data stays close:** embeddings and reranking run locally; document text is not sent out for embedding.
- **Encrypted credentials:** source-database DSNs and provider keys stored with AES-256-GCM, decrypted only at use.
- **Scoped access:** hashed API keys with roles and project membership; project-scoped roles; read-only MCP tokens.
- **Safe by default:** database writes off unless enabled per source, table-whitelisted and fully audited.

Next steps to production

Horizon	Scope (proposed)
Now · production readiness	Security hardening, durable job processing with retries, CI and release automation for every repository, dependency-licence posture
Next · capability depth	NL-to-SQL accuracy benchmarking, table-embedding smart context, richer document coverage, code-graph edges, entity-resolution review UI
Later · breadth and scale	MongoDB connector, writes beyond PostgreSQL, graph community summaries, managed queue and horizontal scale-out

Detail: 02 System Architecture · 03 Infrastructure, Security & Operations · 04 Maturity, Roadmap & IP.